

Briefing on Dynamic Programming Principles and Applications

Executive Summary

Dynamic Programming (DP) is a powerful algorithmic technique for solving optimization problems that exhibit two key properties: **optimal substructure** and **overlapping subproblems**. An optimal substructure means that an optimal solution to a problem contains optimal solutions to its subproblems. Overlapping subproblems implies that a recursive approach would repeatedly solve the same subproblems, creating an opportunity for optimization.

DP leverages these properties using a **tabulation technique**, where intermediate results for subproblems are computed and stored in an array or table. This prevents redundant calculations, often transforming algorithms with exponential complexity into ones with polynomial complexity. Unlike greedy algorithms, which make a locally optimal choice at each step, dynamic programming is more flexible and guarantees an optimal solution for a broader class of problems, including those where a greedy approach fails. Richard Bellman coined the term "dynamic programming," choosing the name partly to secure funding for computational research.

This briefing details the core principles of dynamic programming and examines its application to five distinct problems:

1. **Optimal Change Problem:** Finding the minimum number of coins for a given amount, guaranteed to be optimal for any set of denominations.
2. **Longest Common Subsequence (LCS):** Identifying the longest sequence of elements common to two given sequences, a foundational problem in areas like bioinformatics.
3. **Matrix Chain Multiplication:** Determining the most efficient order to multiply a chain of matrices to minimize the total number of scalar multiplications.
4. **Segmented Least Squares:** Approximating a set of data points with a series of line segments, balancing goodness of fit against the number of segments used.
5. **Maximum Sum of Contiguous Subsequences:** Finding the subarray within a sequence of integers that has the largest sum, demonstrating a linear-time DP solution that vastly outperforms brute-force methods.

1. Core Principles of Dynamic Programming

Dynamic programming is an algorithmic paradigm based on breaking down complex problems into simpler, overlapping subproblems and building up the solution by solving each subproblem only once.

1.1. Foundational Properties

For dynamic programming to be applicable, a problem must possess the following two characteristics:

- **Optimal Substructure:** An optimal solution to the overall problem can be constructed from the optimal solutions of its constituent subproblems. This allows for a recursive or iterative formulation.
- **Overlapping Subproblems:** The space of subproblems must be "small," meaning a recursive algorithm would solve the same subproblems repeatedly. DP exploits this by computing each subproblem's solution just once and storing it in a table (a technique known as tabulation or memoization).

1.2. Comparison with Greedy Algorithms

Greedy algorithms represent a special case of dynamic programming. They make a single, locally optimal choice at each step, hoping it will lead to a globally optimal solution. While efficient, this approach is not always correct. Dynamic programming is more general; it considers all possible choices at each step (by exploring solutions to multiple subproblems) and then selects the one that leads to the best overall solution. This thoroughness allows it to solve complex optimization problems for which a simple greedy strategy fails.

1.3. The Tabulation Method

The standard approach in dynamic programming involves:

1. **Identifying Subproblems:** Defining smaller, manageable versions of the main problem. This often involves varying parameters like the size of an input (e.g., amount of change, length of a sequence).
2. **Formulating a Recurrence:** Establishing a mathematical relation that defines the solution to a problem in terms of the solutions to its subproblems.
3. **Bottom-Up Computation:** Systematically solving the subproblems, starting with the smallest ones, and storing their solutions in a table. The solutions to larger subproblems are then built using the stored results of smaller ones until the solution to the original problem is found.

2. Application: Optimal Change Problem

The optimal change problem seeks to find the minimum number of coins required to make change for a specific amount, given a set of coin denominations. This problem demonstrates a classic case where a greedy algorithm can fail, but dynamic programming provides a universally optimal solution.

- **Problem:** For any given amount c and any set of coin denominations $v[1] > v[2] > \dots > v[n] = 1$, find the minimum number of coins that sum to c .

- **Subproblem:** The (i, j) -problem is defined as finding the minimum number of coins for amount j using only the denominations $v[i], v[i+1], \dots, v[n]$.
- **Tabulation:** A 2D array, $m[i][j]$, is used to store the solution to the (i, j) -problem.
- **Recurrence Relation:** To calculate $m[i][j]$, two cases are considered:
 1. **Do not use coin $v[i]$:** The solution is the same as the optimal solution using the next smaller coins, $m[i+1][j]$.
 2. **Use coin $v[i]$:** The solution is $1 + m[i][j - v[i]]$, representing one coin of value $v[i]$ plus the optimal solution for the remaining amount.
- The final recurrence combines these choices: $m[i][j] = \min\{m[i+1][j], 1 + m[i][j - v[i]]\}$ (if $v[i] \leq j$) $m[i][j] = m[i+1][j]$ (if $v[i] > j$)
- **Algorithm:** The table is filled row by row, starting with the base case where only pennies ($v[n]=1$) are used. For an amount i , this requires i pennies. Subsequent rows introduce larger coins, calculating each entry using the recurrence relation.
- **Example:** For denominations $\{10, 6, 1\}$ and a target amount of 12 cents.
 - A greedy algorithm would choose one 10c coin and two 1c coins, for a total of **3 coins**.
 - Dynamic programming explores all options. The table $m[i][j]$ would be filled as follows:

Coin Set ($v[i] \dots v[n]$)	0	...	5	6	7	...	10	11	12
{1} ($v[3]$)	0	...	5	6	7	...	10	11	12
{6, 1} ($v[2]$)	0	...	5	1	2	...	5	6	2
{10, 6, 1} ($v[1]$)	0	...	5	1	2	...	1	2	2

The DP algorithm correctly identifies that two 6c coins are the optimal solution, using only ****2 coins****. The value ' $m[1][12]$ ' is 2 because ' $m[2][12]$ ' (the solution without using the 10c coin) is 2, which is better than ' $1 + m[1][2]$ ' (using a 10c coin, which would be $1+2=3$ coins).

- **Extension:** To determine the actual coins used, the algorithm can be modified to record which of the two choices in the recurrence led to the minimum value at each step. By backtracking from the final solution, the specific coin combination can be reconstructed.

3. Application: Longest Common Subsequence (LCS)

The LCS problem involves finding a subsequence of maximal length that is common to two given sequences. A subsequence is formed by deleting zero or more elements from a sequence; the remaining elements need not be contiguous.

- **Problem:** Given sequences X of length m and Y of length n , find a common subsequence Z with the maximum possible length.
- **Naive Solution:** A brute-force approach would test every one of the 2^m subsequences of X to see if it is also a subsequence of Y . This leads to an exponential time complexity of $O(n * 2^m)$, making it impractical for all but the shortest sequences.

- **Subproblem:** The subproblem is defined by prefixes of the original sequences. $LCS(x_i, y_j)$ refers to the LCS of the first i elements of x and the first j elements of y .
 - **Tabulation:** A 2D array, $c[i, j]$, stores the *length* of an LCS for prefixes x_i and y_j .
 - **Recurrence Relation:**
 - If $i=0$ or $j=0$, $c[i, j] = 0$ (the LCS of an empty sequence is empty).
 - If $x_i = y_j$, the last elements match and are part of the LCS. The length is one greater than the LCS of the shorter prefixes: $c[i, j] = 1 + c[i-1, j-1]$.
 - If $x_i \neq y_j$, the last elements do not match. The LCS is the longer of the solutions to the two subproblems where one of the mismatched elements is excluded:

$$c[i, j] = \max(c[i-1, j], c[i, j-1])$$
 - **Algorithm:** An $(m+1) \times (n+1)$ table c is created. The first row and column are initialized to 0. The remaining cells are filled in row by row (or column by column) using the recurrence. The final answer for the length is $c[m, n]$. The complexity is $O(m \cdot n)$.
 - **Extension:** To reconstruct the subsequence itself, a second array $b[i, j]$ is used to store which of the three cases in the recurrence yielded the optimal value for $c[i, j]$ (e.g., "diagonal," "up," or "left"). After the table is filled, one can backtrack from $b[m, n]$ to the origin, collecting the characters corresponding to "diagonal" moves to form the LCS.
-

4. Application: Matrix Chain Multiplication

When multiplying a sequence of rectangular matrices, the order of multiplication significantly impacts the total number of scalar operations required, even though the final result is the same due to associativity.

- **Problem:** Given a chain of matrices $A_1, A_2, \dots, A_n$, where A_i has dimensions $d_{i-1} \times d_i$, find the parenthesization that minimizes the total number of scalar multiplications.
- **Naive Solution:** Trying all possible parenthesizations is an exponential problem, related to the Catalan numbers, and thus computationally infeasible.
- **Subproblem:** $M(i, j)$ is defined as the minimum number of scalar multiplications needed to compute the product $A_i * A_{i+1} * \dots * A_j$. The ultimate goal is to find $M(1, n)$.
- **Tabulation:** A 2D array, $M[i, j]$, stores the solution $M(i, j)$.
- **Recurrence Relation:**
 - The base case is $M(i, i) = 0$, as a single matrix requires no multiplication.
 - For a chain $A_i \dots A_j$, we can split it at any point k (where $i \leq k < j$) into $(A_i \dots A_k)$ and $(A_{k+1} \dots A_j)$. The cost is the sum of the costs for computing each sub-product plus the cost of multiplying the two resulting matrices together. The recurrence captures the minimum over all possible split points k :

$$M(i, j) = \min_{\{i \leq k < j\}} \{ M(i, k) + M(k+1, j) + d_{i-1} * d_k * d_j \}$$
- **Algorithm:** The table M is filled along diagonals. The main diagonal $M[i, i]$ is initialized to 0. Then, the first off-diagonal (for chains of length 2) is computed, followed by the second off-diagonal (chains of length 3), and so on, until the top-right corner $M[1, n]$ is

filled. This order ensures that when $M(i, j)$ is calculated, all necessary subproblem solutions have already been computed. The running time is $O(n^3)$.

- **Extension:** To reconstruct the optimal multiplication order, a second array $S[i, j]$ stores the split point k that yielded the minimum cost for $M[i, j]$. A recursive function can then use this S table to print the fully parenthesized expression.

5. Application: Segmented Least Squares

This problem addresses the common statistical challenge of fitting a model to data that is not well-approximated by a single straight line. Instead, it finds an optimal piecewise linear approximation.

- **Problem:** Given a set of n points $P_1, \dots, P_n$, partition them into contiguous segments and find a separate line of best fit for each segment to minimize a total cost function.
- **Cost Function:** The goal is to minimize the sum of two components:
 1. The sum of squared errors (E) for all segments combined.
 2. A penalty for model complexity, calculated as $C * (\text{number of line segments})$, where C is a constant.
- **Subproblem:** $Opt(j)$ is the minimum cost for the first j points ($P_1, \dots, P_j$).
- **Recurrence Relation (Bellman Equation):** The optimal solution for the first j points must end with some final segment $P_i, \dots, P_j$. The algorithm considers all possible start points i for this final segment and chooses the one that minimizes the total cost.
 $Opt(j) = \min_{\{1 \leq i \leq j\}} \{ E_{\{ij\}} + C + Opt(i-1) \}$ where $E_{\{ij\}}$ is the pre-calculated sum of squared errors for a single line fitting points P_i through P_j .
- **Algorithm:**
 1. **Pre-computation:** Calculate the error term $E_{\{ij\}}$ for every possible segment (i, j) . This can take $O(n^3)$ time, but can be optimized to $O(n^2)$ by pre-computing cumulative sums.
 2. **DP Calculation:** Compute $Opt(j)$ for j from 1 to n using the Bellman equation, storing the results. This step takes $O(n^2)$ time.
- **Complexity:** The overall complexity is dominated by the pre-computation step, resulting in $O(n^3)$ or an optimized $O(n^2)$. The constant C allows a user to trade off between accuracy (low error) and parsimony (fewer lines).

6. Application: Maximum Sum of Contiguous Subsequences

This is a frequently asked interview question that elegantly demonstrates the power of dynamic programming to reduce complexity.

- **Problem:** Given an array A of integers (which can be positive, negative, or zero), find the contiguous subsequence whose elements have the largest sum.

- **Naive Solutions:**
 - **Brute Force ($O(n^3)$):** Iterate over all $O(n^2)$ possible start and end indices and sum the elements for each, which takes $O(n)$ time per sum.
 - **Improved with Pre-computation ($O(n^2)$):** Pre-compute an array of running totals. The sum of any subarray can then be found in $O(1)$ time, but checking all $O(n^2)$ subarrays is still required.
- **Subproblem:** $B(b)$ is defined as the maximum sum of a contiguous subsequence that *ends* at index b .
- **Recurrence Relation:** The maximum sum ending at index $b+1$ is either just the element $A[b+1]$ itself (if the best sum ending at b was negative), or it is $A[b+1]$ added to the best sum ending at b . $B(b+1) = \max(A[b+1], B(b) + A[b+1])$
- **Algorithm:**
 1. Create an array B of the same size as A . Initialize $B[0] = A[0]$.
 2. Iterate from $b = 1$ to $n-1$, calculating $B[b]$ using the recurrence.
 3. The final answer is not necessarily $B[n-1]$, but the maximum value found anywhere in the array B , as the overall maximum sum subsequence can end at any index.
- **Complexity:** This dynamic programming solution requires a single pass through the array to populate B and another pass to find the maximum value in B . The overall complexity is $O(n)$, a marked improvement over the quadratic and cubic brute-force solutions.